



Faculty of Engineering
Information security
Laboratory Manual

NAME:

ENROLLMENT NUMBER:

SUBJECT NAME:

SUBJECT CODE: BRANCH:

SEMESTER: ACADEMIC YEAR:



Swaminarayan university
Publication

ISBN Number - 978-81-998475-8-3



सर्व विद्या अधिपतये

Certificate

This is to certify that Mr. / Ms. / Mrs.
having enrollment number of Semester ME /
BE / DIPLOMA in has satisfactorily completed his
term working (Subject name) (subject code:)
during academic year at Faculty of Engineering, Swaminarayan University.

Place:

Date:

ASST. PROF. RADHIKA GOKANI
SUBJECT IN-CHARGE

ASST. PROF. RADHIKA GOKANI
HOD

DR. BHAVNA SONI
DEAN

List of Practicals

Sr. No.	Title	Page No.	Date	Sign
1.	Write a C program to implement Caesar cipher encryption and decryption.			
2.	Write a C program to implement Monoalphabetic cipher encryption-decryption.			
3.	Write a C program to implement Playfair cipher encryption-decryption.			
4.	Write a C program to implement Polyalphabetic cipher encryption-decryption.			
5.	Write a C program to implement Hill cipher encryption decryption.			
6.	Write a C program to implement simple DES			
7.	Write a C program to implement Diffie-Hellman Key exchange Method.			
8.	Write a C program to implement RSA encryption decryption algorithm.			
9.	Write a C program to implement digital signature Algorithm.			
10.	Perform any one of the above encryption technique with cryptool.			
11.	Study and use the Wireshark for the various network protocols.			

Practical -1

Aim : Write a C program to implement Caesar cipher encryption and decryption.

```
#include <stdio.h>
#include <ctype.h>

/* Encrypt a text using Caesar Cipher */
void encrypt(char text[], int key) {
    int i;
    char ch;
    for (i = 0; text[i] != '\0'; i++) {
        ch = text[i];
        if (isupper(ch))
            text[i] = ((ch - 'A' + key) % 26) + 'A';
        else if (islower(ch))
            text[i] = ((ch - 'a' + key) % 26) + 'a';
    }
}

/* Decrypt a text using Caesar Cipher */
void decrypt(char text[], int key) {
    int i;
    char ch;
    for (i = 0; text[i] != '\0'; i++) {
        ch = text[i];
        if (isupper(ch))
            text[i] = ((ch - 'A' - key + 26) % 26) + 'A';
        else if (islower(ch))
            text[i] = ((ch - 'a' - key + 26) % 26) + 'a';
    }
}

int main() {
    char text[100];
    int key;

    printf("Enter text: ");
    fgets(text, sizeof(text), stdin); // Input text
    printf("Enter key (shift value): ");
    scanf("%d", &key);

    encrypt(text, key);
    printf("\nEncrypted text: %s", text);

    decrypt(text, key);
    printf("Decrypted text: %s", text);
}
```

```
    return 0;  
}
```

Input

Enter text: HelloWorld

Enter key (shift value): 3

Output

Encrypted text: KoorZruog

Decrypted text: HelloWorld

Practical -2

Aim : Write a C program to implement Monoalphabetic cipher encryption-decryption.

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

/* Encryption and Decryption keys */
char encryptionKey[26] = "QWERTYUIOPASDFGHJKLZXCVBNM"; // Example key
char decryptionKey[26]; // Will be generated based on encryption key

/* Function to generate decryption key from encryption key */
void generateDecryptionKey() {
    int i;
    for (i = 0; i < 26; i++) {
        decryptionKey[encryptionKey[i] - 'A'] = 'A' + i;
    }
}

/* Encrypt the plaintext */
void encrypt(char text[]) {
    int i;
    for (i = 0; text[i] != '\0'; i++) {
        char ch = text[i];
        if (isupper(ch))
            text[i] = encryptionKey[ch - 'A'];
        else if (islower(ch))
            text[i] = tolower(encryptionKey[ch - 'a']);
    }
}

/* Decrypt the ciphertext */
void decrypt(char text[]) {
    int i;
    for (i = 0; text[i] != '\0'; i++) {
        char ch = text[i];
        if (isupper(ch))
            text[i] = decryptionKey[ch - 'A'];
        else if (islower(ch))
            text[i] = tolower(decryptionKey[ch - 'a']);
    }
}

int main() {
    char text[100];
```

```
generateDecryptionKey();

printf("Enter text: ");
fgets(text, sizeof(text), stdin); // Read text
text[strcspn(text, "\n")] = '\0'; // Remove newline if present

encrypt(text);
printf("\nEncrypted text: %s\n", text);

decrypt(text);
printf("Decrypted text: %s\n", text);

return 0;
}
```

Input

Enter text: HelloWorld

Output

Encrypted text: ItssgVgksr

Decrypted text: HelloWorld

Practical -3

Aim : Write a C program to implement Playfair cipher encryption-decryption.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char keyMatrix[5][5];

/* Remove duplicate letters and replace J with I */
void prepareKey(char key[]) {
    int i, j, k = 0;
    int alpha[26] = {0};
    char temp[25];
    for (i = 0; key[i] != '\0'; i++) {
        char ch = toupper(key[i]);
        if (ch == 'J') ch = 'I';
        if (ch < 'A' || ch > 'Z') continue;
        if (!alpha[ch - 'A']) {
            temp[k++] = ch;
            alpha[ch - 'A'] = 1;
        }
    }
    // Fill the rest of the letters
    for (i = 0; i < 26; i++) {
        if (!alpha[i] && i + 'A' != 'J') temp[k++] = i + 'A';
    }
    // Fill key matrix
    k = 0;
    for (i = 0; i < 5; i++)
        for (j = 0; j < 5; j++)
            keyMatrix[i][j] = temp[k++];
}

/* Find position of a character in key matrix */
void findPosition(char ch, int *row, int *col) {
    int i, j;
    if (ch == 'J') ch = 'I';
    for (i = 0; i < 5; i++)
        for (j = 0; j < 5; j++)
            if (keyMatrix[i][j] == ch) {
                *row = i;
                *col = j;
                return;
            }
}
```

```
/* Encrypt plaintext using Playfair cipher */
void encrypt(char text[]) {
    int i = 0;
    while (text[i]) {
        if (!isalpha(text[i])) { i++; continue; }
        char a = toupper(text[i]);
        char b;
        if (text[i+1]) {
            b = toupper(text[i+1]);
            if (!isalpha(b)) {
                b = 'X';
            } else if (a == b) {
                b = 'X';
            } else i++;
        } else {
            b = 'X';
        }

        int r1, c1, r2, c2;
        findPosition(a, &r1, &c1);
        findPosition(b, &r2, &c2);

        if (r1 == r2) {
            text[i] = keyMatrix[r1][(c1 + 1) % 5];
            text[i+1] = keyMatrix[r2][(c2 + 1) % 5];
        } else if (c1 == c2) {
            text[i] = keyMatrix[(r1 + 1) % 5][c1];
            text[i+1] = keyMatrix[(r2 + 1) % 5][c2];
        } else {
            text[i] = keyMatrix[r1][c2];
            text[i+1] = keyMatrix[r2][c1];
        }
        i += 2;
    }
}
```

```
/* Decrypt ciphertext using Playfair cipher */
void decrypt(char text[]) {
    int i = 0;
    while (text[i]) {
        if (!isalpha(text[i])) { i++; continue; }
        char a = toupper(text[i]);
        char b = toupper(text[i+1]);

        int r1, c1, r2, c2;
        findPosition(a, &r1, &c1);
        findPosition(b, &r2, &c2);
```

```
    if (r1 == r2) {
        text[i] = keyMatrix[r1][(c1 + 4) % 5]; // shift left
        text[i+1] = keyMatrix[r2][(c2 + 4) % 5];
    } else if (c1 == c2) {
        text[i] = keyMatrix[(r1 + 4) % 5][c1]; // shift up
        text[i+1] = keyMatrix[(r2 + 4) % 5][c2];
    } else {
        text[i] = keyMatrix[r1][c2];
        text[i+1] = keyMatrix[r2][c1];
    }
    i += 2;
}
}
```

```
int main() {
    char key[25], text[100];

    printf("Enter key: ");
    fgets(key, sizeof(key), stdin);
    key[strcspn(key, '\n')] = '\0';

    printf("Enter text: ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, '\n')] = '\0';

    prepareKey(key);

    encrypt(text);
    printf("\nEncrypted text: %s\n", text);

    decrypt(text);
    printf("Decrypted text: %s\n", text);

    return 0;
}
```

Input

Enter key: MONARCHY

Enter text: HELLO

Output

Encrypted text: BMNDZ

Decrypted text: HELLO

Practical -4

Aim : Write a C program to implement Polyalphabetic cipher encryption-decryption.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Function to encrypt text using Vigenère Cipher */
void encrypt(char text[], char key[]) {
    int i, j = 0;
    int keyLen = strlen(key);

    for (i = 0; text[i] != '\0'; i++) {
        char ch = text[i];
        if (isalpha(ch)) {
            char k = toupper(key[j % keyLen]) - 'A';
            if (isupper(ch))
                text[i] = ((ch - 'A' + k) % 26) + 'A';
            else
                text[i] = ((ch - 'a' + k) % 26) + 'a';
            j++;
        }
    }
}

/* Function to decrypt text using Vigenère Cipher */
void decrypt(char text[], char key[]) {
    int i, j = 0;
    int keyLen = strlen(key);

    for (i = 0; text[i] != '\0'; i++) {
        char ch = text[i];
        if (isalpha(ch)) {
            char k = toupper(key[j % keyLen]) - 'A';
            if (isupper(ch))
                text[i] = ((ch - 'A' - k + 26) % 26) + 'A';
            else
                text[i] = ((ch - 'a' - k + 26) % 26) + 'a';
            j++;
        }
    }
}

int main() {
    char text[100], key[100];
```

```
printf("Enter text: ");
fgets(text, sizeof(text), stdin);
text[strcspn(text, "\n")] = '\0'; // Remove newline

printf("Enter key: ");
fgets(key, sizeof(key), stdin);
key[strcspn(key, "\n")] = '\0'; // Remove newline

encrypt(text, key);
printf("\nEncrypted text: %s\n", text);

decrypt(text, key);
printf("Decrypted text: %s\n", text);

return 0;
}
```

Input

Enter text: HELLOWORLD

Enter key: KEY

Output

Encrypted text: RIJVSUYVJN

Decrypted text: HELLOWORLD

Practical -5

Aim : Write a C program to implement Hill cipher encryption decryption.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Function to calculate modular inverse */
int modInverse(int a, int m) {
    a = a % m;
    for (int x = 1; x < m; x++)
        if ((a * x) % m == 1)
            return x;
    return -1;
}

/* Function to encrypt 2 letters using Hill Cipher */
void encrypt(char text[], int key[2][2], int n) {
    int i;
    for (i = 0; i < n; i += 2) {
        int x1 = toupper(text[i]) - 'A';
        int x2 = toupper(text[i+1]) - 'A';

        int y1 = (key[0][0]*x1 + key[0][1]*x2) % 26;
        int y2 = (key[1][0]*x1 + key[1][1]*x2) % 26;

        text[i] = y1 + 'A';
        text[i+1] = y2 + 'A';
    }
}

/* Function to decrypt 2 letters using Hill Cipher */
void decrypt(char text[], int key[2][2], int n) {
    int det = key[0][0]*key[1][1] - key[0][1]*key[1][0];
    det = (det % 26 + 26) % 26;

    int invDet = modInverse(det, 26);
    if (invDet == -1) {
        printf("Key matrix not invertible modulo 26.\n");
        return;
    }

    int invKey[2][2];
    invKey[0][0] = (key[1][1] * invDet) % 26;
    invKey[0][1] = ((-key[0][1] + 26) * invDet) % 26;
```

```
invKey[1][0] = ((-key[1][0] + 26) * invDet) % 26;
invKey[1][1] = (key[0][0] * invDet) % 26;

int i;
for (i = 0; i < n; i += 2) {
    int x1 = text[i] - 'A';
    int x2 = text[i+1] - 'A';

    int y1 = (invKey[0][0]*x1 + invKey[0][1]*x2) % 26;
    int y2 = (invKey[1][0]*x1 + invKey[1][1]*x2) % 26;

    text[i] = (y1 + 26) % 26 + 'A';
    text[i+1] = (y2 + 26) % 26 + 'A';
}

int main() {
    char text[100];
    int key[2][2];
    int n, i;

    printf("Enter 4 letters of key matrix (A-Z):\n");
    for (i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++) {
            char ch;
            scanf(" %c", &ch);
            key[i][j] = toupper(ch) - 'A';
        }

    printf("Enter plaintext (even number of letters): ");
    scanf("%s", text);
    n = strlen(text);

    if (n % 2 != 0) {
        text[n] = 'X'; // Padding if needed
        text[n+1] = '\0';
        n++;
    }

    encrypt(text, key, n);
    printf("Encrypted text: %s\n", text);

    decrypt(text, key, n);
    printf("Decrypted text: %s\n", text);

    return 0;
}
```

Input

Enter 4 letters of key matrix (A-Z):

G H

I J

Enter plaintext (even number of letters): HELLO

Output

Encrypted text: XUBYLL

Decrypted text: HELLOX

Practical -6

Aim : Write a C program to implement simple DES.

```
#include <stdio.h>
#include <string.h>

/* Permutation functions */
int permute(int input, int perm[], int n) {
    int output = 0;
    for (int i = 0; i < n; i++) {
        int bit = (input >> (7 - perm[i])) & 1;
        output = (output << 1) | bit;
    }
    return output;
}

/* Simple S-Boxes for demonstration */
int sbox0[4][4] = {{1,0,3,2},{3,2,1,0},{0,2,1,3},{3,1,3,2}};
int sbox1[4][4] = {{0,1,2,3},{2,0,1,3},{3,0,1,0},{2,1,0,3}};

/* Function to apply S-box substitution (4-bit input -> 2-bit output) */
int sbox(int input, int box[4][4]) {
    int row = ((input & 0b1000)>>2) | (input & 0b0001);
    int col = (input & 0b0110)>>1;
    return box[row][col];
}

/* Simple F function for S-DES */
int F(int right, int subkey) {
    int expanded = ((right & 0b1111)<<1) | ((right & 0b1000)>>3); // Expand 4->5 bits for example
    int xored = expanded ^ subkey;
    int left2 = (xored & 0b1110)>>2;
    int right2 = xored & 0b0011;
    int s0 = sbox(left2, sbox0);
    int s1 = sbox(right2, sbox1);
    return (s0<<2) | s1;
}

/* Encrypt 8-bit plaintext using 8-bit key (simplified DES) */
int encrypt(int plaintext, int key) {
    int L = (plaintext & 0b11110000)>>4;
    int R = plaintext & 0b00001111;

    int subkey1 = (key & 0b11110000)>>4;
    int subkey2 = key & 0b00001111;
```

```
int temp = R;
R = L ^ F(R, subkey1);
L = temp;

temp = R;
R = L ^ F(R, subkey2);
L = temp;

return (L<<4) | R;
}

/* Decrypt 8-bit ciphertext using 8-bit key */
int decrypt(int ciphertext, int key) {
    int L = (ciphertext & 0b11110000)>>4;
    int R = ciphertext & 0b00001111;

    int subkey1 = (key & 0b11110000)>>4;
    int subkey2 = key & 0b00001111;

    int temp = R;
    R = L ^ F(R, subkey2);
    L = temp;

    temp = R;
    R = L ^ F(R, subkey1);
    L = temp;

    return (L<<4) | R;
}

int main() {
    int plaintext, key;

    printf("Enter 8-bit plaintext (0-255): ");
    scanf("%d", &plaintext);
    printf("Enter 8-bit key (0-255): ");
    scanf("%d", &key);

    int ciphertext = encrypt(plaintext, key);
    printf("Encrypted text: %d\n", ciphertext);

    int decrypted = decrypt(ciphertext, key);
    printf("Decrypted text: %d\n", decrypted);

    return 0;
}
```

Input

Enter 8-bit plaintext (0-255): 100

Enter 8-bit key (0-255): 45

Output

Encrypted text: 187

Decrypted text: 100

Practical -7

Aim : Write a C program to implement Diffie-Hellmen Key exchange Method.

```
#include <stdio.h>
#include <math.h>

/* Function to calculate (base^exp) % mod using modular exponentiation */
long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        exp = exp >> 1;
        base = (base * base) % mod;
    }
    return result;
}

int main() {
    long long p, g; // public prime and primitive root
    long long a, b; // private keys of Alice and Bob
    long long A, B; // public keys of Alice and Bob
    long long keyAlice, keyBob; // shared secret key

    printf("Enter a prime number (p): ");
    scanf("%lld", &p);
    printf("Enter a primitive root modulo p (g): ");
    scanf("%lld", &g);

    printf("Enter private key for Alice (a): ");
    scanf("%lld", &a);
    printf("Enter private key for Bob (b): ");
    scanf("%lld", &b);

    // Compute public keys
    A = mod_pow(g, a, p); // Alice's public key
    B = mod_pow(g, b, p); // Bob's public key

    printf("\nPublic Key of Alice: %lld\n", A);
    printf("Public Key of Bob: %lld\n", B);

    // Compute shared secret keys
    keyAlice = mod_pow(B, a, p); // Alice computes (B^a) % p
    keyBob = mod_pow(A, b, p); // Bob computes (A^b) % p
```

```
printf("\nShared Secret Key (Alice): %lld\n", keyAlice);
printf("Shared Secret Key (Bob): %lld\n", keyBob);

if (keyAlice == keyBob)
    printf("\nKey exchange successful! Shared secret established.\n");
else
    printf("\nKey exchange failed!\n");

return 0;
}
```

Input

Enter a prime number (p): 23
Enter a primitive root modulo p (g): 5
Enter private key for Alice (a): 6
Enter private key for Bob (b): 15

Output

Public Key of Alice: 8
Public Key of Bob: 19

Shared Secret Key (Alice): 2
Shared Secret Key (Bob): 2

Key exchange successful! Shared secret established.

Practical -8

Aim : Write a C program to implement RSA encryption decryption algorithm.

```
#include <stdio.h>
#include <math.h>

/* Function to calculate gcd */
int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

/* Function for modular exponentiation: (base^exp) % mod */
long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        exp = exp >> 1;
        base = (base * base) % mod;
    }
    return result;
}

/* Function to calculate modular inverse using brute-force (small numbers) */
int modInverse(int e, int phi) {
    int d = 1;
    while ((e * d) % phi != 1)
        d++;
    return d;
}

int main() {
    int p, q;
    printf("Enter prime number p: ");
    scanf("%d", &p);
    printf("Enter prime number q: ");
    scanf("%d", &q);

    int n = p * q;
```

```
int phi = (p - 1) * (q - 1);

int e;
// Choose e such that 1 < e < phi and gcd(e, phi) = 1
for (e = 2; e < phi; e++) {
    if (gcd(e, phi) == 1)
        break;
}

int d = modInverse(e, phi);

printf("\nPublic Key (n, e): (%d, %d)\n", n, e);
printf("Private Key (n, d): (%d, %d)\n", n, d);

char message[100];
printf("\nEnter message to encrypt (capital letters only, no spaces): ");
scanf("%s", message);

// Encryption
int i;
long long cipher[100];
for (i = 0; message[i] != '\0'; i++) {
    int m = message[i] - 'A';
    cipher[i] = mod_pow(m, e, n);
}
int len = i;

printf("\nEncrypted message: ");
for (i = 0; i < len; i++)
    printf("%lld ", cipher[i]);

// Decryption
char decrypted[100];
for (i = 0; i < len; i++)
    decrypted[i] = mod_pow(cipher[i], d, n) + 'A';
decrypted[len] = '\0';

printf("\nDecrypted message: %s\n", decrypted);

return 0;
}
```

Input

Enter prime number p: 3

Enter prime number q: 11

Enter message to encrypt (capital letters only, no spaces): HELLO

Output

Public Key (n, e): (33, 3)

Private Key (n, d): (33, 7)

Encrypted message: 17 29 25 25 11

Decrypted message: HELLO

Practical -9

Aim : Write a C program to implement digital signature Algorithm.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/* Function to calculate (base^exp) % mod */
long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        exp = exp >> 1;
        base = (base * base) % mod;
    }
    return result;
}

/* Function to calculate gcd */
int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

/* Function to calculate modular inverse using brute-force (small numbers) */
int modInverse(int a, int mod) {
    int x;
    for (x = 1; x < mod; x++)
        if ((a * x) % mod == 1)
            return x;
    return -1;
}

int main() {
    long long p, q, g, x, y, k;
    long long m, r, s;

    srand(time(0));
```

```

// Public parameters (small numbers for demo)
printf("Enter prime p (public): ");
scanf("%lld", &p);
printf("Enter prime q (divides p-1, public): ");
scanf("%lld", &q);
printf("Enter g (public, 1 < g < p): ");
scanf("%lld", &g);

// Private key x (1 < x < q)
printf("Enter private key x (1 < x < q): ");
scanf("%lld", &x);

// Compute public key y = g^x mod p
y = mod_pow(g, x, p);
printf("Public key y = %lld\n", y);

// Message to sign
printf("Enter message (integer) m: ");
scanf("%lld", &m);

// Choose random k (1 < k < q, gcd(k,q)=1)
do {
    k = rand() % (q - 2) + 1; // 1 < k < q
} while (gcd(k, q) != 1);

// Compute r = (g^k mod p) mod q
r = mod_pow(g, k, p) % q;

// Compute s = (k^-1 * (m + x*r)) mod q
long long k_inv = modInverse(k, q);
s = (k_inv * (m + x * r)) % q;

printf("\nSignature: (r, s) = (%lld, %lld)\n", r, s);

// Verification
long long w = modInverse(s, q);
long long u1 = (m * w) % q;
long long u2 = (r * w) % q;
long long v = ((mod_pow(g, u1, p) * mod_pow(y, u2, p)) % p) % q;

printf("Verification v = %lld\n", v);

if (v == r)
    printf("Signature is valid!\n");
else
    printf("Signature is invalid!\n");

```

```
    return 0;  
}
```

Input

Enter prime p (public): 23
Enter prime q (divides $p-1$, public): 11
Enter g (public, $1 < g < p$): 2
Enter private key x ($1 < x < q$): 6
Enter message (integer) m : 9

Output

Public key $y = 9$
Signature: $(r, s) = (6, 8)$
Verification $v = 6$
Signature is valid!

Practical -10

Aim : Perform any one of the above encryption technique with cryptool.

Step-by-Step Guide: Caesar Cipher in CrypTool 2

Step 1: Open CrypTool

1. Launch **CrypTool 2**.
2. Go to **File** → **New** to start a new project.

Step 2: Choose Caesar Cipher

1. Click **Algorithms** in the toolbar.
2. Select **Classical** → **Caesar Cipher**.

Step 3: Enter Plaintext

1. In the **Input field**, type the plaintext you want to encrypt.
Example: HELLOWORLD

Step 4: Set Key

1. Set the **shift value** (key) in the key field.
Example: 3 → Each letter will shift by 3 positions.

Step 5: Encrypt

1. Click **Encrypt** (or press the play button in the workflow).
2. The encrypted text will appear in the **Output field**.
Example:
 3. Input: HELLOWORLD
 4. Key: 3
 5. Output: KHOORZRUOG

Step 6: Decrypt

1. To verify, click **Decrypt** with the same key.
2. You should get back the original plaintext: HELLOWORLD.

Step 7: Optional – Experiment

- Try **different keys** and **messages**.
- CrypTool allows visualization of the **encryption process**, which is helpful for learning.

Practical -11

Aim : Study and use the Wireshark for the various network protocols.

Wireshark: Study and Use Guide

1. Install Wireshark

1. Download from <https://www.wireshark.org/>.
2. Install with default options. On Windows, install **Npcap** when prompted for packet capture.

2. Capture Network Traffic

1. Open Wireshark.
2. Select the **network interface** connected to the internet (e.g., Ethernet, Wi-Fi).
3. Click **Start Capturing Packets** (green shark fin).
4. Let it run while you perform network activities (browsing, ping, etc.).
5. Click **Stop** when done.

3. Filter Packets by Protocol

Use **display filters** to study specific protocols:

Protocol	Filter	What to Observe
IP	ip	Source/destination IP addresses, TTL, protocol type (TCP/UDP).
TCP	tcp	Handshake (SYN, SYN-ACK, ACK), sequence/ack numbers, flags.
UDP	udp	Source/destination ports, length, payload.
HTTP	http	GET/POST requests, URLs, response codes.
HTTPS/TLS	tls	Encrypted traffic, handshake, certificates.
DNS	dns	Query/response, domain names, resolved IPs.
ICMP	icmp	Ping requests/replies, TTL, echo sequence.
ARP	arp	MAC address resolution, who-has/reply.

Example: To view only TCP traffic:

tcp

To view only HTTP requests:

http.request

4. Analyze Packets

1. Click a packet in the list.
2. Expand **Packet Details** pane to see header fields.
 - Example: For TCP → Source Port, Destination Port, Flags, Sequence Number.
3. Expand **Packet Bytes** to see raw hex and ASCII data.

5. Follow a Stream

- Right-click a packet → **Follow** → **TCP Stream** or **UDP Stream**.
- This reconstructs the conversation between client and server (useful for HTTP or chat analysis).

6. Statistics

- **Statistics** → **Protocol Hierarchy** → Shows which protocols are most used.
- **Statistics** → **Conversations** → Shows which IPs communicate and how much data.
- **Statistics** → **IO Graphs** → Graph packet rate over time.

7. Save & Export

- Save captured packets as .pcap for future study.
- Export packet bytes or summary for reports.

Tips for Effective Study

1. Generate traffic for different protocols:
 - **HTTP:** Visit websites
 - **DNS:** Ping domain names
 - **ICMP:** Ping IP addresses
 - **ARP:** Turn Wi-Fi on/off
2. Observe **handshakes** for TCP.
3. Compare **UDP vs TCP** behavior.
4. Try **filtering by IP addresses** to isolate conversations.